Vector Query Language (VQL) - Complete Specification

Overview

Vector Query Language (VQL) is a SQL-inspired query language designed for vector databases. It combines the familiar syntax of SQL with powerful vector operations, making it intuitive for developers while providing advanced capabilities for machine learning and AI applications.

Core Concepts

Data Model

Collection: Top-level container for vector data (analogous to a database schema)
Table: Structured set of vectors with associated metadata (analogous to a database table)
Vector: Numerical array with fixed dimensions
Embedding: Specialized vector representing an encoded entity
Metadata: Additional structured information stored as native columns

Basic Syntax Structure

```sql
SELECT fields
FROM collection_name.table_name
VECTOR_OPERATION operation_parameters
WHERE metadata_filters
ORDER BY distance_metric
LIMIT top_k;
```

Vector Operations

1. Similarity Search

```sql
```

```sql
-- Basic similarity search
SELECT *
FROM ecommerce.product_vectors
SIMILARITY SEARCH [1.2, 0.8, -0.2, 0.5]
USING METRIC cosine
TOP K 10;

-- With metadata filtering
SELECT product_name, category, price
FROM ecommerce.product_vectors
SIMILARITY SEARCH [1.2, 0.8, -0.2, 0.5]
USING METRIC euclidean
WHERE category = 'electronics' AND price < 1000
TOP K 5
THRESHOLD 0.8;
```

2. Hybrid Search

```sql
-- Combining vector similarity with text search
HYBRID SEARCH (
  VECTOR [1.2, 0.8, -0.2] WEIGHT 0.7,
  TEXT "machine learning" WEIGHT 0.3
)
FROM research.paper_vectors
WHERE publication_year > 2020;
```

3. Range Search

```sql
-- Find all vectors within a distance threshold
SELECT *
FROM user_data.behavior_vectors
RANGE SEARCH [user_vector]
USING METRIC cosine
THRESHOLD 0.5;
```

4. Batch Operations

```sql
```

```sql
-- Batch similarity search
BATCH SIMILARITY SEARCH (
  SELECT query_vectors FROM user_queries.batch_requests
)
FROM ecommerce.product_vectors
TOP K 5;
```

Linear Transformations

1. Matrix Operations

```sql
-- Define transformation matrices
CREATE TRANSFORM rotation_matrix AS MATRIX [
  [cos(theta), -sin(theta)],
  [sin(theta), cos(theta)]
];

-- Apply transformations
SELECT TRANSFORM(vector_data, rotation_matrix) AS rotated_vector
FROM machine_learning.feature_vectors;

-- Chain multiple transformations
SELECT TRANSFORM_CHAIN(
  vector_data,
  [scaling_matrix, rotation_matrix, projection_matrix]
) AS transformed_vector
FROM machine_learning.feature_vectors;
```

2. Built-in Linear Transformations

```sql
```

```sql
-- Principal Component Analysis
SELECT PCA(vector_data, dimensions=50) AS reduced_vector
FROM analytics.high_dimensional_data;

-- Random Projection
SELECT RANDOM_PROJECTION(vector_data, output_dim=100, seed=42) AS projected
FROM analytics.sparse_vectors;

-- Whitening Transform
SELECT WHITEN(vector_data) AS whitened_vector
FROM analytics.correlated_data;

-- Normalization
SELECT NORMALIZE(vector_data) AS unit_vector
FROM analytics.unnormalized_data;
```

3. Linear Algebra Operations

```sql
-- Matrix multiplication
SELECT MATMUL(matrix1, matrix2) AS product;

-- Eigendecomposition
SELECT EIGENDECOMPOSE(correlation_matrix) AS (
  eigenvalues,
  eigenvectors
);

-- Singular Value Decomposition
SELECT SVD(data_matrix) AS (U, S, V);
```

Nonlinear Transformations

1. Hyperbolic Space Operations

```sql
```

```sql
-- Project vectors into Poincaré ball model
SELECT POINCARE_PROJECTION(vector_data, radius=1.0) AS hyperbolic_vector
FROM hierarchy.tree_structures;

-- Hyperbolic distance calculation
SELECT HYPERBOLIC_DISTANCE(
  vector1,
  vector2,
  model='poincare'
) AS h_distance;

-- Möbius addition in hyperbolic space
SELECT MOBIUS_ADD(
  hyperbolic_vec1,
  hyperbolic_vec2,
  curvature=-1.0
) AS combined_vector;

-- Exponential and logarithmic maps
SELECT EXPMAP(tangent_vector, base_point, model='poincare') AS hyperbolic_point;
SELECT LOGMAP(hyperbolic_point, base_point, model='poincare') AS tangent_vector;
```

2. Manifold Operations

sql

```sql
-- Riemannian optimization on manifold
OPTIMIZE ON MANIFOLD 'hyperbolic'
OBJECTIVE minimize_hyperbolic_distance(source_vectors, target_vectors)
USING riemannian_gradient_descent(learning_rate=0.01, max_iterations=1000);

-- Parallel transport along geodesics
SELECT PARALLEL_TRANSPORT(
  vector,
  start_point,
  end_point,
  manifold='poincare'
) AS transported_vector;

-- Geodesic interpolation
SELECT GEODESIC_INTERPOLATE(
  point1,
  point2,
  t=0.5,
  manifold='poincare'
) AS midpoint;
```

3. Advanced Nonlinear Transforms

```sql
-- Kernel transformations
SELECT KERNEL_TRANSFORM(
  vector_data,
  kernel='rbf',
  gamma=0.1
) AS kernel_space;

-- Manifold learning
SELECT ISOMAP(vector_data, n_neighbors=5, n_components=2) AS manifold_vector;
SELECT UMAP(vector_data, n_neighbors=15, min_dist=0.1) AS umap_vector;
SELECT TSNE(vector_data, perplexity=30, n_components=2) AS tsne_vector;
```

4. Space Conversions

```sql
```

```sql
-- Convert between hyperbolic models
SELECT CONVERT_SPACE(
  hyperbolic_vector,
  from='poincare',
  to='lorentz'
) AS lorentz_vector;

-- Project between different curvatures
SELECT CURVATURE_PROJECT(
  vector_data,
  source_curvature=-1.0,
  target_curvature=-0.5
) AS projected;
```

Vector Functions and Aggregations

1. Vector Functions

sql

```sql
-- Basic vector operations
SELECT DIMENSION(embedding) AS vector_dim;
SELECT DISTANCE(vector1, vector2, 'cosine') AS similarity;
SELECT CONCAT(vector1, vector2) AS combined_vector;
SELECT DOT(vector1, vector2) AS dot_product;

-- Vector arithmetic
SELECT vector_data + influence_vector AS adjusted_vector;
SELECT vector_data * scalar_value AS scaled_vector;
```

2. Vector Aggregations

sql

```sql
-- Compute centroids
SELECT AVG(user_vector) AS centroid
FROM analytics.user_profiles
GROUP BY user_category;

-- Find representative vectors
SELECT MEDIAN_VECTOR(feature_vector) AS representative
FROM analytics.product_features
GROUP BY product_category;
```

Clustering and Analytics

1. Clustering Operations

```sql
-- K-means clustering
CLUSTER customer_data.behavior_vectors
USING KMEANS
CLUSTERS 5
OUTPUT AS customer_segments;

-- Hierarchical clustering
CLUSTER content.document_vectors
USING HIERARCHICAL
LINKAGE 'ward'
OUTPUT AS doc_hierarchy;
```

2. Quality Control and Sampling

```sql
-- Diversity sampling
SELECT *
FROM search.result_vectors
SIMILARITY SEARCH [query_vector]
WITH DIVERSITY FACTOR 0.3
TOP K 10;

-- Outlier detection
SELECT *, OUTLIER_SCORE(behavior_vector) AS anomaly_score
FROM analytics.user_behavior
WHERE OUTLIER_SCORE(behavior_vector) > 0.95;
```

Index Management

1. Index Creation

```sql
```

```sql
-- Create vector index with specific parameters
CREATE INDEX product_vectors_idx
ON ecommerce.product_vectors
USING ALGORITHM 'hnsw'
WITH (
  M = 16,
  ef_construction = 200,
  distance_metric = 'cosine'
);

-- Create composite index for filtered searches
CREATE INDEX category_vector_idx
ON ecommerce.product_vectors (category, vector_data)
USING ALGORITHM 'ivfflat'
WITH (lists = 1000);
```

2. Index Optimization

```sql
-- Rebuild index with new parameters
REBUILD INDEX product_vectors_idx
WITH (M = 32, ef_construction = 400);

-- Analyze index performance
ANALYZE INDEX product_vectors_idx;
```

Advanced Features

1. Custom Transform Functions

```sql
```

```sql
-- Define custom transformation
CREATE TRANSFORM normalize_and_scale AS (
  SELECT NORMALIZE(vector_data) * scale_factor AS result
  WHERE scale_factor = 1.5
);

-- Composite transformations
CREATE TRANSFORM feature_extractor AS
  CHAIN(
    NORMALIZE(),
    PCA(dimensions=50),
    POINCARE_PROJECTION(radius=1.0)
  );
```

2. Optimization and Learning

```sql
-- Learn optimal transformations
OPTIMIZE TRANSFORM learning_projection
OBJECTIVE minimize_distance(source_vectors, target_vectors)
USING gradient_descent(learning_rate=0.01, max_iterations=1000);

-- Validate transformations
VALIDATE TRANSFORM my_transform
CHECK (
  orthogonality = TRUE,
  determinant != 0,
  condition_number < 100
);
```

Type System

1. Vector Types

```sql
VECTOR(dim)          -- Fixed-dimension vector
DYNAMIC_VECTOR       -- Variable-dimension vector
EMBEDDING            -- Specialized vector with metadata
HYPERBOLIC_VECTOR(model, dim)  -- Vector in hyperbolic space
MANIFOLD_POINT(type, coords)   -- Point on a specific manifold
```

2. Transform Types

```sql
MATRIX(rows, cols)     -- Fixed-dimension matrix
TRANSFORM              -- Function type for transformations
TRANSFORM_CHAIN        -- Composite transformation
DISTANCE               -- Numeric type for similarity scores
```

3. Space Definitions

```sql
-- Define custom spaces
CREATE SPACE hyperbolic_space (
  model = 'poincare',
  curvature = -1.0,
  dimension = 50
);

-- Validate constraints
VALIDATE IN hyperbolic_space
CHECK (
  point_norm < 1.0,
  satisfies_hyperbolic_constraints = TRUE
);
```

Error Handling and Validation

1. Dimension Handling

```sql
-- Handle dimension mismatches
ON DIMENSION MISMATCH
  THEN PAD_ZEROS | TRUNCATE | REJECT;

-- Invalid vector handling
ON INVALID VECTOR
  THEN NULL | REJECT | DEFAULT [default_vector];
```

2. Quality Assurance

```sql
```

```sql
-- Validate vector quality
SELECT *
FROM analytics.user_data
WHERE VECTOR_QUALITY(user_vector) > 0.9;

-- Check for NaN or infinite values
SELECT *
FROM analytics.processed_data
WHERE IS_VALID_VECTOR(vector_data) = TRUE;
```

Performance Considerations

1. Query Optimization

Lazy evaluation of vector operations
Automatic batching for large-scale operations
Index-aware query planning
Parallel processing capabilities

2. Caching Strategies

Frequently accessed vector caching
Transform result caching
Query plan caching

3. Memory Management

Streaming processing for large datasets
Efficient vector storage formats
Garbage collection for temporary vectors

Implementation Guidelines

1. Backend Integration

Support for multiple vector database backends
Pluggable distance metric implementations
Extensible transformation framework

2. API Design

RESTful API for remote query execution
Streaming results for large result sets
Async query execution capabilities

3. Security and Access Control

Role-based access to collections and tables
Query-level security policies
Vector data encryption support

Example Use Cases

1. Recommendation Systems

```sql
SELECT product_id, product_name, similarity_score
FROM ecommerce.product_vectors
SIMILARITY SEARCH (
  SELECT user_vector FROM analytics.user_profiles WHERE user_id = 12345
)
WHERE category IN ('electronics', 'books')
TOP K 20;
```

2. Semantic Search

```sql
SELECT document_id, title, relevance
FROM content.document_vectors
HYBRID SEARCH (
  VECTOR [text_vector] WEIGHT 0.8,
  TEXT "artificial intelligence" WEIGHT 0.2
)
WHERE publication_date > '2020-01-01'
TOP K 50;
```

3. Anomaly Detection

```sql
SELECT user_id, behavior_vector, anomaly_score
FROM analytics.user_behavior
WHERE OUTLIER_SCORE(behavior_vector) > 0.95
ORDER BY anomaly_score DESC;
```

4. Hierarchical Clustering

```sql
SELECT product_id, category,
    HYPERBOLIC_HIERARCHY_EMBED(feature_vector, depth=3) AS hierarchical_pos
FROM ecommerce.product_features
ORDER BY HYPERBOLIC_TREE_DISTANCE(hierarchical_pos, root_category);
```

This specification provides a comprehensive foundation for implementing vector query capabilities while maintaining SQL-like familiarity and extending into advanced mathematical operations.